Cyprus University of Technology

CSE 227 Προχωρημένη Αντικειμενοστρεφής Μεθοδολογία και Προγραμματισμός (UML, Java)

Constructors, equals, toString
Αντικείμενα ως παράμετροι

Παναγιώτης Ηλία

Constructors (Κατασκευαστές)

- Ο Constructor είναι μια «μέθοδος» η οποία καλείται όταν δημιουργούμε το αντικείμενο χρησιμοποιώντας την new
- Αν δεν έχουμε ορίσει Constructor καλείται ένας default constructor χωρίς ορίσματα που δεν κάνει τίποτα
- Αν ορίσουμε constructor, τότε καλείται ο constructor που ορίσαμε
  - Ο default constructor αντικαθίσταται – δεν μπορεί πλέον να χρησιμοποιηθεί (έκτος και αν τον ορίσουμε)

Παράδειγμα

```
class Person
{
    private String name;

    public Person(String name){
       this.name = name;
    }

    public void speak(String s){
       System.out.println(name+": "+s);
    }
}

public class HelloWorld2
{
    public static void main(String[] args){
       Person alice = new Person("Alice");
       alice.speak("Hello World");
    }
}
```

Constructor: μια μέθοδος με το ίδιο όνομα όπως και η κλάση και χωρίς τύπο (ούτε void)

Αρχικοποιεί την μεταβλητή name

Constructor: καλείται όταν δημιουργείται το αντικείμενο με την new

Παράδειγμα

- Μία κλάση που να αποθηκεύει ημερομηνίες
  - Η κλάση θα παίρνει την ημέρα, μήνα και χρόνο σαν νούμερα (π.χ., 20 3 2024) και θα μπορεί να τυπώνει την ημερομηνία με το όνομα του μήνα (π.χ., 20 Μαρτίου 2024)
  - Στο πρόγραμμα να βάλετε μια ημερομηνία και να την τυπώσετε

```
class Date
{
        private int day = 1;
        private int month = 1;
        private int year = 2000;
        private String[] monthNames = {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                       "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

        public Date(int day, int month, int year)
        {
                if (day <= 0 || day > 31 || month <= 0 || month >12 ){
                        return;
                }
                this.day = day;
                this.month = month;
                this.year = year;
        }

        public void printDate(){
                System.out.println(day + " " + monthNames[month-1] + " " + year);
        }
}

class DateExample
{
        public static void main(String args[])
        {
                Date myDate = new Date(20,3,2024);
                myDate.printDate();
        }
}
```

```
class Date
{
  private int day; private int month; private int year;
  private String[] monthNames =
          {"Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov","Dec"};

  public Date(int day, int month, int year)
  {
    if (checkDay(day)){this.day = day;}
    if (checkMonth(month)){this.month = month;}
    this.year = year;
  }

  private boolean checkDay(int day){
     if (day <= 0 || day > 31 ) {return false;}
     return true;
  }

  private boolean checkMonth(int day){
     if (month <= 0 || month >12) {return false;}
     return true;
  }

  public void printDate()
  {
    System.out.println(day + " " + monthNames[month-1] + " " + year);
  }
}
```

Ο constructor μπορεί να καλεί και άλλες μεθόδους που κάνουν κάποια από τη δουλειά που χρειάζεται

```
class Date
{
        private int day = 1;
        private int month = 1;
        private int year = 2000;
        private String[] monthNames = {"Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                       "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"};

        public Date(int day, int month, int year)
        {
                if (day <= 0 || day > 31 || month <= 0 || month >12 ){
                        return;
                }
                this.day = day;
                this.month = month;
                this.year = year;
        }

        public void printDate(){
                System.out.println(day + " " + monthNames[month-1] + " " + year);
        }
}

class DateExample
{
        public static void main(String args[])
        {
                Date myDate = new Date(20,3,2024);
                myDate.printDate();
        }
}
```

Η εκτέλεση αυτών των αρχικοποιήσεων γίνεται πριν εκτελεστούν οι εντολές στον constructor

Τι γίνεται αν μπούμε στο if?

Παραδείγματα

- Θέλουμε μια κλάση Student που να κρατάει πληροφορίες για έναν φοιτητή. Τι πεδία πρέπει να έχουμε? Τι θα μπει στον constructor?

- Θέλουμε μια κλάση GuestList που να χειρίζεται τους καλεσμένους σε ένα πάρτι. Τι πεδία πρέπει να έχουμε? Πώς θα κάνουμε τον constructor?

```
class Student
{
	private String name = "John Doe";
	private int AM = 1000;


	public Student(String name, int AM){
		this.name = name;
		this. AM = AM;
	}

	public void printInfo(){
		System.out.println(name + " " + AM);
	}

	public static void main(String[] args){
		Student aStudent = new Student("Kostas", 1001);
		aStudent.printInfo();
	}
}
```

Guest List

```
class GuestList
{
        private String[] names;
        private boolean[] confirm;
        int numberOfGuests;

        public GuestList(int numberOfGuests)
        {
                this.numberOfGuests = numberOfGuests;
                names = new String[numberOfGuests];
                confirm = new boolean[numberOfGuests];
                // Εδώ μπορούμε να έχουμε κώδικα για τις τιμές
                // Ή να εισάγονται τα ονόματα ένα ένα.
        }

}
```

Δεσμεύει μνήμη για τους πίνακες με τα ονόματα των καλεσμένων και τις επιβεβαιώσεις

Υπερφόρτωση (Overloading)

- Η Java μας δίνει τη δυνατότητα να ορίσουμε πολλές μεθόδους με το ίδιο όνομα μέσω της διαδικασίας της υπερφόρτωσης (overloading)
  - Ορισμός πολλών μεθόδων με το ίδιο όνομα αλλά διαφορετικά ορίσματα, μέσα στην ίδια κλάση

```
class Car
{
        private int position;

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar9
{
        public static void main(String args[]){
          Car myCar = new Car(1);
          myCar.move();
          myCar.move(-1);
        }
}
```

Μετακινεί το όχημα μια θέση μπροστά

Μετακινεί το όχημα μια θέση πίσω

Υπογραφή Μεθόδου

- Η υπογραφή μίας μεθόδου είναι το όνομα της και η λίστα με τους τύπους των ορισμάτων της μεθόδου
  - Η Java μπορεί να ξεχωρίσει μεθόδους με διαφορετική υπογραφή
  - Π.χ., move(), move(int) έχουν διαφορετική υπογραφή
  - Ο τύπος του return **δεν** είναι μέρος της υπογραφής

Constructor overloading

```
class Car
{
        private int position;

        public Car(){
                this.position = 0;
        }

        public Car(int position){
                this.position = position;
        }

        public void move(){
                position ++ ;
        }

        public void move(int delta){
                position += delta ;
        }

}

class MovingCar10
{
        public static void main(String args[]){
                Car myCar1 = new Car(1); myCar1.move();
                Car myCar2= new Car(); myCar2.move(-1);
        }
}
```

Υπερφόρτωση - Προσοχή

- Όταν ορίζουμε ένα constructor, ο default constructor παύει να υπάρχει. Πρέπει να τον ορίσουμε μόνοι μας
- Η υπερφόρτωση γίνεται μόνο ως προς τα ορίσματα, ΟΧΙ ως προς την επιστρεφόμενη τιμή
- Λόγω της συμβατότητας μεταξύ τύπων μια κλήση μπορεί να ταιριάζει με διάφορες μεθόδους. Καλείται αυτή που ταιριάζει ακριβώς, ή αυτή που είναι πιο κοντά
- Αν υπάρχει ασάφεια στο ποια συνάρτηση πρέπει να κληθεί θα χτυπήσει ο compiler

Αντικείμενα ως Ορίσματα

- Μπορούμε να περνάμε **αντικείμενα ως ορίσματα** σε μία μέθοδο όπως οποιαδήποτε άλλη μεταβλητή
- Οποιαδήποτε κλάση μπορεί να χρησιμοποιηθεί ως παράμετρος
- Όταν τα ορίσματα ανήκουν στην κλάση στην οποία ορίζεται η μέθοδος τότε η μέθοδος μπορεί να δει (και) τα ιδιωτικά (private) πεδία των αντικειμένων
- Αν τα ορίσματα είναι διαφορετικού τύπου τότε η μέθοδος μπορεί μόνο να καλέσει τις public μεθόδους

Διάβασμα Πεδίων

- Η προσπέλαση των πεδίων (για διάβασμα ή γράψιμο) γίνεται με τον ίδιο τρόπο όπως και η προσπέλαση των μεθόδων

```
<όνομα αντικειμένου>.<όνομα πεδίου>
```

```
class Car
{
  private int position = 0;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += delta ;
  }

  public int distanceFrom(Car other){
    return this.position - other.position;
  }
}

class MovingCarDistance2
{
  public static void main(String args[]){
    Car myCar1 = new Car(1);
    Car myCar2 = new Car(0); myCar2.move(2);
    System.out.println("Distance of Car 1 from Car 2: " + myCar1.distanceFrom(myCar2));
    System.out.println("Distance of Car 2 from Car 1: " + myCar2.distanceFrom(myCar1));
}
```

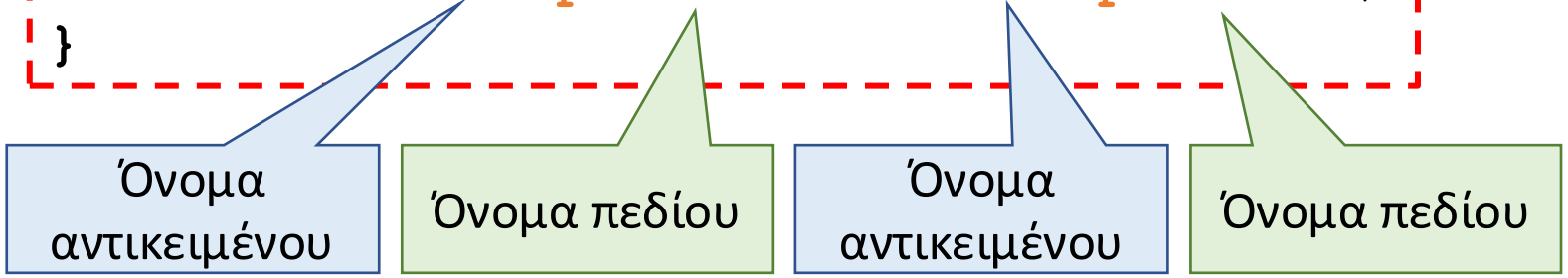
Στο σημείο αυτό διαβάζουμε τα πεδία position για το αντικείμενο this και other. Αν και το πεδίο position είναι private μπορούμε να το προσπελάσουμε γιατί είμαστε μέσα στην κλάση Car

Διάβασμα Πεδίων

- Η προσπέλαση των πεδίων (για διάβασμα ή γράψιμο) γίνεται με τον ίδιο τρόπο όπως και η προσπέλαση των μεθόδων
- Και το αντικείμενο this είναι μια τέτοια περίπτωση

```
<όνομα αντικειμένου>.<όνομα πεδίου>
```

```
public int distanceFrom(Car other){
    return this.position - other.position;
}
```



Παράδειγμα

- Ορίστε μια μέθοδο που θα παίρνει όρισμα ένα άλλο όχημα και θα βάζει το όχημα που είναι πιο πίσω στην ίδια θέση με το όχημα που είναι πιο μπροστά

```
class Car
{
   private int position = 0;

   public Car(){}

   public void move(int delta){
      position += delta ;
   }

   public void catchUp(Car other){
      if (this.position > other.position){
         other.position = this.position;
      }else{
         this.position = other.position;
      }
   }

   public void printPosition(){
      System.out.println("Car is at position "+position);
   }
}

class MovingCar13{
   public static void main(String args[]){
      Car myCar1 = new Car(); myCar1.move(10);
      Car myCar2= new Car(); myCar2.move(20);
      myCar1.printPosition(); myCar2.printPosition();
      myCar1.catchUp(myCar2);
      myCar1.printPosition(); myCar2.printPosition();
   }
}
```

Μπορούμε όχι μόνο να διαβάσουμε αλλά και να αλλάξουμε την τιμή του πεδίου position στο αντικείμενο other

Δυο ειδικές μέθοδοι

- Η Java «περιμένει» να δει τις εξής δύο μεθόδους για κάθε αντικείμενο
  - Τη μέθοδος toString η οποία για ένα αντικείμενο επιστρέφει μία string αναπαράσταση του αντικειμένου
  - Τη μέθοδο equals η οποία ελέγχει για ισότητα δύο αντικειμένων
- Και οι δύο συναρτήσεις ορίζονται από τον προγραμματιστή
  - Τι String θα επιστραφεί και τι σημαίνει δύο αντικείμενα να είναι ίσα μπορούν να οριστούν όπως μας βολεύει

Παράδειγμα

- Στην κλάση Car θέλουμε να προσθέσουμε τις μεθόδους toString και equals
  - Η toString θα επιστρέφει ένα String με τη θέση του αυτοκινήτου
  - Η equals θα ελέγχει αν δύο οχήματα έχουν την ίδια θέση

toString()

```
class Car
{
  private Integer position = 0;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += delta ;
  }

  public String toString(){
    return position.toString();
  }
}
class MovingCarToString
{
  public static void main(String args[]){
    Car myCar1 = new Car(1);
    Car myCar2 = new Car(0); myCar2.move(2);
    System.out.println("Car 1 is at " + myCar1 + " and car 2 is at " + myCar2);
  }
}
```

Για να μπορούμε να μετατρέψουμε τον ακέραιο σε String ορίζουμε το position ως Integer (wrapper class)

Η Java περιμένει αυτό το συντακτικό για τον ορισμό της toString

Μετά καλούμε τη συνάρτηση toString() της κλάσης Integer

Χρησιμοποιούμε τις myCar1,myCar2 σαν String. Καλείται η μέθοδος toString() αυτόματα

Ισοδύναμο με το:

```
System.out.println("Car 1 is at " + myCar1.toString() + " and car 2 is at " + myCar2.toString());
```

toString()

```
class Car
{
  private int position = 0;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += delta ;
  }

  public String toString(){
    return " "+position;
  }
}

class MovingCarToString
{
  public static void main(String args[]){
    Car myCar1 = new Car(1);
    Car myCar2 = new Car(0); myCar2.move(2);
    System.out.println("Car 1 is at " + myCar1 + " and car 2 is at " + myCar2);
  }
}
```

Ένας άλλος τρόπος να μετατρέψουμε ένα int σε String

```
class Car
{
  private int position = 0;

  public Car(int position){
    this.position = position;
  }

  public void move(int delta){
    position += delta ;
  }

  public boolean equals(Car other){
    if (this.position == other.position){
      return true;
    }
    return false;
  }
}

class MovingCarEquals
{
  public static void main(String args[]){
    Car myCar1 = new Car(2);
    Car myCar2 = new Car(0); myCar2.move(2);
    if (myCar1.equals(myCar2)){
       System.out.println("Collision!");
    }
  }
}
```

Η Java περιμένει αυτό το συντακτικό για τον ορισμό της equals

Ένα παράδειγμα αντικειμένου ως παράμετρος συνάρτησης

Αν και το πεδίο position είναι private μπορούμε να το προσπελάσουμε γιατί είμαστε μέσα στην κλάση Car.

Κλήση της equals στο πρόγραμμα

Παράδειγμα

- Πως θα ορίσουμε τις μεθόδους toString και equals για την κλάση Person?

```
class Person
{
    private String name;

    public Person(String name){
        this.name = name;
    }

    public String toString(){
        return name;
    }

    public boolean equals(Person other){
        return this.name.equals(other.name);
    }

}

public class TwoPersons
{
    public static void main(String[] args){
        Person alice = new Person("Alice");
        Person bob = new Person("Bob");
        if (!alice.equals(bob)){
            System.out.println("There are two different persons: "+ alice + "and " + bob);
        }
    }
}
```

```
class Person{
    private String firstName;
    private String lastName;

    public Person(String firstName, String lastName){
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public String toString(){
        return firstName + " " + lastName;
    }

    public boolean equals(Person other){
        return (this.firstName.equals(other.firstName))
                && (this.lastName.equals(other.lastName));
    }

}

public class TwoPersons2
{
    public static void main(String[] args){
        Person alice = new Person("Alice", "Wonderland");
        Person bob = new Person("Bob", "Sfougkarakis");
        if (!alice.equals(bob)){
            System.out.println("There are two different persons: "
                                                + alice + "and " + bob);
        }
    }
}
```

toString και equals

- Η μέθοδος toString ορίζεται πάντα ως:

```
public String toString(){
    …
}
```

- Αν έχουμε ορίσει την toString μπορούμε να χρησιμοποιήσουμε τα αντικείμενα της κλάσης σαν Strings
  - Καλείτε αυτόματα η toString
- Η μέθοδος equals ορίζεται πάντα ως:

```
public boolean equals(<Class name> other){
    …
}
```

Αντικείμενα σαν Ορίσματα – Παράδειγμα Ι

- Θέλουμε να προσομοιώσουμε την κυκλοφορία σε ένα δρόμο
  - Έχουμε ένα φανάρι που μπορεί να είναι πράσινο, ή κόκκινο. Αλλάζει σε κάθε βήμα
  - Έχουμε ένα αυτοκίνητο που σε κάθε βήμα κινείται μία θέση, αν το φανάρι δεν είναι κόκκινο
- Κλάσεις:
  - TrafficLight: κρατάει την κατάσταση του φαναριού και αλλάζει την κατάσταση του
  - Car: Τροποποίηση της move ώστε παίρνει όρισμα το φανάρι και να κινείται μόνο αν το φανάρι δεν είναι κόκκινο
  - TrafficSimulation: κάνει την προσομοίωση

```
class TrafficLight
{
  boolean isLightRed = false;

  public void change(){
    isLightRed = !isLightRed;
  }

  public boolean isRed(){
    return isLightRed;
  }

  public void printStatus(){
    if (isLightRed){
       System.out.println(
         "Traffic light is red");
    }else{
       System.out.println(
         "Traffic light is green");
    }
}
```

```
class Car
{
   private int position = 0;

   public int printPosition() {
      System.out.println("Car at "+ position);
   }

   public void move(TrafficLight light){
      if (!light.isRed()){
         position ++;
      }
   }
}
```

```
class TrafficSimulation
{
   public static void main(String[] args){
      TrafficLight light = new TrafficLight();
      Car myCar = new Car();
      for (int i = 0; i < 10; i ++){
          light.printStatus();
          myCar.printPosition();
          myCar.move(light);
          light.change();
      }
   }
}
```

Αντικείμενα σαν Ορίσματα – Παράδειγμα ΙΙ

- Η κλάση Car θα έχει ως πεδίο και το όνομα του οδηγού. Το όνομα θα το παίρνει από ένα αντικείμενο της κλάσης Person στην αρχικοποίηση.

```
class Person
{
  private String name;

  public Person(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }
}
```

```
class Car
{
  private int position = 0;
  private String driverName;

  public Car(int position, Person driver){
    this.position = position;
    driverName = driver.getName();
  }

  public String toString(){
    return driverName + " " + position;
  }
}
```

```
class MovingCarDriver
{
  public static void main(String args[])
  {
    Person alice = new Person("Alice");
    Car myCar = new Car(1, alice);
    System.out.println(myCar);
  }
}
```

Αντικείμενα μέσα σε αντικείμενα

- Εκτός από ορίσματα σε μεθόδους, αντικείμενα οποιασδήποτε κλάσης μπορούν να εμφανιστούν και ως πεδία μιας κλάσης
  - Ένα αντικείμενο μπορεί να έχει μέσα του άλλα αντικείμενα

```
class Person
{
  private String name;

  public Person(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, String name){
    this.position = position;
    this.driver = new Person(name);
  }

  public String toString(){
    return driver.getName() + " " + position;
  }
}
```

```
class MovingCarDriver
{
  public static void main(String args[])
  {
    Car myCar = new Car(1, "Alice");
    System.out.println(myCar);
  }
}
```

Το αντικείμενο δημιουργείται μέσα στον constructor

Αυτό έχει νόημα αν το Person χρησιμοποιείται μόνο μέσα στην κλάση Car

```
class Person
{
  private String name;

  public Person(String name){
    this.name = name;
  }

  public String getName(){
    return name;
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, Person driver){
    this.position = position;
    this.driver = driver;
  }

  public String toString(){
    return driver.getName()+ " " + position;
  }
}
```

```
class MovingCarDriver
{
  public static void main(String args[])
  {
    Person alice = new Person("Alice");
    Car myCar = new Car(1, alice);
    System.out.println(myCar);
  }
}
```

Καλύτερη υλοποίηση!

```
class Person
{
  private String name;
  private int age;

  public Person(String name, int age){
    this.name = name;
    this.age = age;
  }

  public String getName(){
    return name;
  }

  public int getAge(){
    return age;
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, Person driver){
    this.position = position;
    if (driver.getAge() >= 18){
        this.driver = driver;
    }
  }

  public String toString(){
    return driver.getName() + " " + position;
  }
}
```

Η Person είναι διαφορετική κλάση άρα δεν μπορούμε να διαβάσουμε το πεδίο age

```
class MovingCarDriver
{
  public static void main(String args[])
  {
    Person alice = new Person("Alice", 20);
    Car myCar = new Car(1, alice);
    System.out.println(myCar);
  }
}
```

Κώδικας σε πολλά αρχεία

- Όταν έχουμε πολλές κλάσεις βολεύει να τις βάζουμε σε διαφορετικά αρχεία
  - Το κάθε αρχείο έχει το όνομα της κλάσης

- Ένα επιπλέον πλεονέκτημα είναι ότι μπορούμε να ορίσουμε μια main συνάρτηση για κάθε κλάση ξεχωριστά
  - Βοηθάει για το testing του κώδικα
- Για να κάνουμε compile πολλά αρχεία μαζι:
  - `javac file1.java file2.java file3.java`
    - Ή μπορούμε να κάνουμε compile το “βασικό” αρχείο
      - (e.g MovingCarDriver.java slide 38)

Η εντολή exit

Χρησιμοποιείται για σοβαρά λάθη για να σταματάει την εκτέλεση του προγράμματος.

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, Person driver){
    this.position = position;
    if (driver.getAge() >= 18){
        this.driver = driver;
    }
    else{
       System.exit(-1);
    }
  }
}
```

Αν δώσουμε μη αποδεκτή ηλικία το πρόγραμμα μας θα σταματήσει.

Το -1 εξυπηρετεί σαν κωδικός λάθους, μπορείτε να βάλετε όποια τιμή θέλετε.

```
class Person
{
  private String name;
  private int licence;

  public Person(String name, int licence){
      this.name = name;
      this.licence = licence;
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, Person driver){
    this.position = position;
    this.driver = driver;
  }
}
```

Πως θα υλοποιήσουμε την toString και την equals?

```
class Person
{
  private String name;
  private int licence;

  public Person(String name,int licence){
    this.name = name;
    this.licence = licence;
  }

  public String toString(){
    return name + " " + licence;
  }

  public boolean equals(Person other){
    if (this.name.equals(other.name)&&
      this.licence == other.licence)){
        return true
    }else{
        return false;
    }
  }
}
```

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, Person driver){
    this.position = position;
    this.driver = driver;
  }

  public String toString(){
    return driver + " " + position;
  }

  public boolean equals(Car other){
    if (this.position == other.position &&
        this.driver.equals(other.driver)){
        return true;
    }else{
        return false;
    }
}
```

Φωλιασμένη κλήση της toString και της equals

Παράδειγμα

- Φτιάξετε μια κλάση που να χειρίζεται ένα λογαριασμό τράπεζας. Κρατάει το όνομα του ιδιοκτήτη και το ποσό
- Δημιουργείστε και μία μέθοδο που συγχωνεύει δύο λογαριασμούς του ίδιου ατόμου

```
class BankAccount
{
	private String name;
	private int amount;

	public BankAccount(String name, int amount){
		this.name = name;
		this.amount = amount;
	}

	public void merge(BankAccount other){
		if (this.name.equals(other.name)){this.amount += other.amount;
		}
	}
}
```

Είναι σύνηθες το αποτέλεσμα μιας μεθόδου να αποθηκεύει το αποτέλεσμα της στο ίδιο αντικείμενο το οποίο κάλεσε την μέθοδο

Π.χ. Εδώ το αποτέλεσμα της συγχώνευσης αποθηκεύεται στον λογαριασμό που έκανε την κλήση

Αντικείμενα ως επιστρεφόμενες τιμές

- Μία μέθοδος μπορεί να επιστρέφει αντικείμενα όπως οποιαδήποτε άλλη τιμή
- Είναι δυνατόν επίσης μέσα σε μία μέθοδο να δημιουργούμε ένα αντικείμενο και να το επιστρέφουμε για να χρησιμοποιηθεί μετά

```
class Car
{
  private int position = 0;
  private Person driver;

  public Car(int position, String name){
    this.position = position;
    this.driver = new Person(name);
  }

  public String toString(){
    return driver.getName()+ " " + position;
  }

  public Person getDriver(){
    return driver;
  }
}
```

Επιστρέφει το αντικείμενο Person το οποίο είναι ο οδηγός του οχήματος

```
class BankAccount
{
    private String name;
    private int amount;

    public BankAccount(String name, int amount){
        this.name = name;
        this.amount = amount;
    }

    public void merge(BankAccount other){
        if (this.name.equals(other.name)){
            this.amount += other.amount;
        }
    }

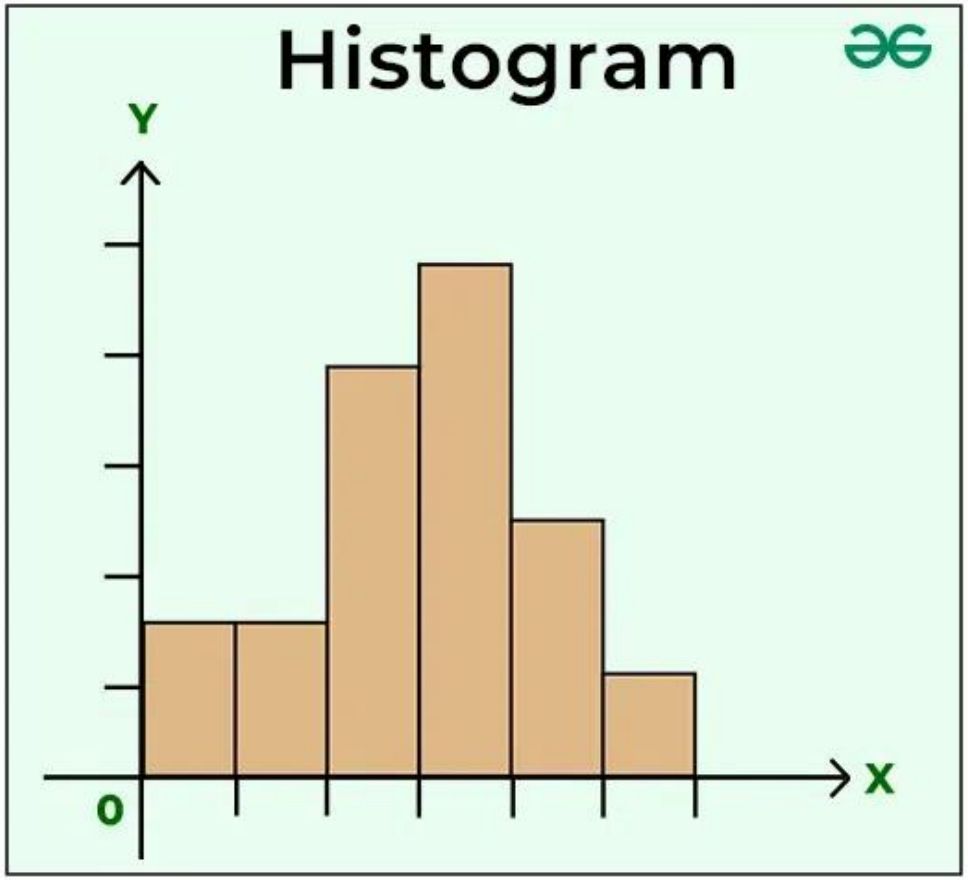
    public BankAccount mergeIntoNewAccount(BankAccount other){
        if (this.name.equals(other.name)){
            BankAccount newAccount =
                        new BankAccount(name,this.amount+other.amount);
            return newAccount;
        }
        return null;
    }
}
```

Μια άλλη επιλογή είναι να δημιουργήσουμε ένα νέο λογαριασμό μετά την συγχώνευση

Δημιουργούμε ένα νέο αντικείμενο BankAccount και το επιστρέφουμε

Αν δεν μπορούμε να δημιουργήσουμε το νέο λογαριασμό επιστρέφουμε null. Το null είναι το κενό αντικείμενο

Παράδειγμα: Histogram



- Ένα ιστόγραμμα μετράει για ένα σύνολο από τιμές πόσες φορές εμφανίζεται η κάθε τιμή
  - Για παράδειγμα αν έχω τις τιμές: 1,2,1,2,4,5,3,3,3,2,4 το ιστόγραμμα τους είναι 2,3,3,2,1, και είναι ο αριθμός εμφανίσεων των τιμών 1,2,3,4,5 αντίστοιχα
- Να υλοποιήσουμε μια κλάση **GradeHistogram** η οποία κρατάει ένα ιστόγραμμα για τους βαθμούς ενός μαθήματος
  - Η κλάση θα κρατάει το μέγιστο βαθμό για το μάθημα, και ένα πίνακα με τον αριθμό εμφανίσεων του κάθε βαθμού. Αν ο μέγιστος βαθμός είναι maxGrade τότε οι πιθανοί βαθμοί θα είναι όλοι οι ακέραιοι στο διάστημα [1,maxGrade]
  - Η κλάση θα πρέπει να έχει και τις εξής μεθόδους:
    1. Ένα **constructor**, ο οποίος θα παίρνει σαν όρισμα τον μέγιστο βαθμό και ένα πίνακα με βαθμούς και δημιουργεί το ιστόγραμμα των βαθμών
    2. Μια μέθοδο **toString**, η οποία θα επιστρέφει ένα String που αναπαριστά το ιστόγραμμα. Για το ιστόγραμμα στο παραπάνω παράδειγμα θα επιστρέφει το String: «1:2 2:3 3:3 4:2 5:1»
    3. Την μέθοδο **equals**, η οποία θα συγκρίνει αν δύο ιστογράμματα είναι ίδια

Παράδειγμα: Histogram

- Τι πληροφορία (δεδομένα) θέλουμε να κρατάει η κλάση μας?
  - Το μέγιστο βαθμό
  - Τις τιμές του ιστογράμματος
- Η πληροφορία (τα δεδομένα) που θέλουμε να κρατάει η κλάση θα είναι τα πεδία της κλάσης
  - Έναν ακέραιο maxGrade με το μέγιστο βαθμό (που θα είναι και το μήκος του πίνακα)
  - Ένα πίνακα ακεραίων histogram με τις συχνότητες για τον κάθε βαθμό

Κατασκευή Ιστογράμματος

- Πως φτιάχνουμε ένα ιστόγραμμα από ένα πίνακα με βαθμούς?
  - Κάθε φορά που βλέπουμε τον βαθμό x θα πρέπει να αυξήσουμε την x-θέση του ιστογράμματος κατά ένα

```
for (int i = 0; i < grades.length; i ++){
      int x = grades[i];
      histogram[x-1] ++;
}
```

Η μέθοδος toString

- Στην μέθοδο **toString** πρέπει να δημιουργήσουμε το String το οποίο θα αναπαριστά το ιστόγραμμα.
- Για να το κάνουμε αυτό θα πρέπει να διατρέξουμε τον πίνακα με τις τιμές και να φτιάξουμε το String αυξητικά

```
public String toString(){
    String output = "";
    for (int i = 0; i < maxGrade; i ++){
      output = output + (i+1) + ":" + histogram[i] + " ";
    }
    return output;
}
```

Σωστό ή λάθος?

```
class GradeHistogram
{

  public GradeHistogram(int maxGrade, int[] grades)
  {
       int[] histogram = new int[maxGrade];
       for (int i = 0; i < grades.length; i ++){
            int x = grades[i];
            histogram[x-1] ++;
       }
  }

  public String toString(){
  {
       String output = "";
       for (int i = 0; i < maxGrade; i ++){
            output = output + (i+1) + ":" + histogram[i] + " ";
       }
       return output;
  }
}
```

Οι μεταβλητές **maxGrade** και **histogram** δεν είναι ορισμένες.

Για να μπορεί να τις βλέπει η μέθοδος toString (ή οποιαδήποτε άλλη μέθοδος) θα πρέπει να είναι ορισμένες ως πεδία της κλάσης.

ΛΑΘΟΣ!

Σωστό ή λάθος?

```
class GradeHistogram
{
  private int maxGrade;
  private int[] histogram;

  public GradeHistogram(int maxGrade, int[] grades)
  {
        int[] histogram = new int[maxGrade];
        for (int i = 0; i < grades.length; i ++){
              int x = grades[i];
              histogram[x-1] ++;
        }
  }

  public String toString(){
  {
        String output = "";
        for (int i = 0; i < maxGrade; i ++){
              output = output + (i+1) +
                    ":" + histogram[i] + " ";
        }
        return output;
  }
}
```

Ο constructor δεν αρχικοποιεί τα πεδία της κλάσης.

Οι μεταβλητές maxGrade και histogram που ορίζονται μέσα στον constructor είναι τοπικές μεταβλητές και δεν αλλάζουν την τιμή των πεδίων

ΛΑΘΟΣ!

Σωστό ή λάθος?

```
class GradeHistogram
{
  private int maxGrade;
  private int[] histogram;

  public GradeHistogram(int maxGrade, int[] grades)
  {
      this.maxGrade = maxGrade;
      for (int i = 0; i < grades.length; i ++){
            int x = grades[i];
            histogram[x-1] ++;
      }
  }

  public String toString(){
  {
      String output = "";
      for (int i = 0; i < maxGrade; i ++){
            output = output + (i+1) + ":" + histogram[i] + " ";
      }
      return output;
  }
}
```

Η μεταβλητή maxGrade αρχικοποιείται σωστά.

Ο πίνακας histogram όμως όχι.

Τον έχουμε ορίσει σωστά αλλά δεν τον έχουμε δημιουργήσει (δεν του έχουμε δώσει χώρο)! Δεν έχουμε προσδιορίσει το μέγεθος του.

ΛΑΘΟΣ!

Σωστό ή λάθος?

```
class GradeHistogram
{
  private int maxGrade;
  private int[] histogram = new int[maxGrade];

  public GradeHistogram(int maxGrade, int[] grades
  {
        this.maxGrade = maxGrade;
        for (int i = 0; i < grades.length; i ++){
              int x = grades[i];
              histogram[x-1] ++;
        }
  }

  public String toString(){
  {
        String output = "";
        for (int i = 0; i < maxGrade; i ++){
              output = output + (i+1) +
                    ":" + histogram[i] + " ";
        }
        return output;
  }
}
```

Οι εντολές αυτές θα εκτελεστούν πριν από τις εντολές του constructor. Εκείνη τη στιγμή δεν ξέρουμε το μέγιστο βαθμό και άρα δημιουργούμε ένα πίνακα μηδενικού μεγέθους!

ΛΑΘΟΣ!

```
class GradeHistogram
{
  private int maxGrade;
  private int[] histogram;

  public GradeHistogram(int maxGrade, int[] grades)
  {
        histogram = new int[maxGrade];
        for (int i = 0; i < grades.length; i ++){
              int x = grades[i];
              histogram[x-1] ++;
        }
  }

  public String toString(){
  {
        String output = "";
        for (int i = 0; i < maxGrade; i ++){
              output = output + (i+1) +
                    ":" + histogram[i] + " ";
        }
        return output;
  }
}
```

Σωστό ή λάθος?

Ο Constructor θα αρχικοποιήσει σωστά τον πίνακα histogram, αλλά δεν θα αλλάξει το πεδίο maxGrade μιας και χρησιμοποιεί την τοπική μεταβλητή – παράμετρο.

Το maxGrade εδώ αναφέρεται στο πεδίο και έχει τιμή μηδέν.

ΛΑΘΟΣ!

Πρώτα δηλώνουμε τα πεδία μέσα στην κλάση.

Στον Constructor δίνουμε τιμή στο maxGrade και αφού πλέον ξέρουμε το μήκος του πίνακα τον δημιουργούμε και του δίνουμε χώρο για να κρατάει τις τιμές.

```
class GradeHistogram
{
   private int maxGrade;
   private int[] histogram;

   public GradeHistogram(int maxGrade, int[] grades)
   {
        this.maxGrade = maxGrade;
        histogram = new int[maxGrade];
        for (int i = 0; i < grades.length; i ++){
              int x = grades[i];
              histogram[x-1] ++;
        }
   }

   public String toString()
   {
        String output = "";
        for (int i = 0; i < maxGrade; i ++){
              output = output + (i+1) +
                    ":" + histogram[i] + " ";
        }
        return output;
   }
}
```

ΣΩΣΤΟ!

Εμβέλεια Μεταβλητών

- Η κάθε μεταβλητή έχει εμβέλεια μέσα στο block στο οποίο ορίζεται
  - Τις μεταβλητές-πεδία της κλάσης μπορούν να τις χρησιμοποιήσουν όλες οι μέθοδοι της κλάσης
    - Οι μεταβλητές έχουν ζωή όσο υπάρχει το αντίστοιχο αντικείμενο της κλάσης
  - Οι μεταβλητές που ορίζονται μέσα σε μία μέθοδο μπορούν να χρησιμοποιηθούν μόνο μέσα στη μέθοδο
    - Οι μεταβλητές χάνονται όταν βγούμε από τη μέθοδο
  - Οι παράμετροι μιας μεθόδου είναι σαν τοπικές μεταβλητές της μεθόδου

Η μέθοδος equals

```
public boolean equals(GradeHistogram other)
{
      if (this.maxGrade != other.maxGrade){
            return false;
      }
      for (int i = 0; i < maxGrade; i ++){
            if (this.histogram[i] != other.histogram[i]){
                  return false;
            }
      }
      return true;
 }
```

Δεν κάνουμε έλεγχο ισότητας χρησιμοποιώντας την toString!
Το String που επιστρέφουμε μπορεί να μην ικανοποιεί τον έλεγχο ισότητας

Είναι πιο εύκολο να ελέγξουμε για την περίπτωση της ανισότητας παρά της ισότητας. Μόλις μία από τις συνθήκες δεν ικανοποιείται επιστρέφουμε false. Αν φτάσουμε μέχρι τέλους ικανοποιούνται όλες και άρα επιστρέφουμε true